

AI Usage Report (PA1 – PA5 Final)

Project: LoveYou – AI-Enhanced Dating Web Application
Course: Software Engineering (CS300 / CSC13002 - SE24C11 - Group 01)
Sprint: PA5-2026 (Final Project Submission)
Submission Date: 23/08/2026

1. AI Usage Declaration & Scope

Performed by: Nghĩa | **Reviewed by:** Chiến | **Edited by:** Nghĩa

Throughout the entire development lifecycle of the LoveYou project (spanning Sprint 1 to Sprint 5 / PA1 to PA5), Group 01 utilized Artificial Intelligence (AI) tools to assist with specification generation, system architecture modeling, code implementation, test scenario formulation, and documentation formatting.

The team explicitly declares that:

- AI tools were employed strictly as assistive productivity multipliers and brainstorming aids, not as autonomous replacements for human engineering analysis, critical thinking, or decision-making.
- All AI-generated suggestions, code snippets, architectural diagrams, and test scenarios were rigorously reviewed, manually verified, debugged, and integrated by team members.
- The project team maintains 100% intellectual ownership and full accountability for the complete source code, architectural integrity, test outcomes, and project documentation submitted in this package.

2. AI Tools & Models Catalogue

Performed by: Chiến, Hoàng | **Reviewed by:** Nghĩa | **Edited by:** Chiến

The following AI tools and platforms were utilized across the project:

AI Tool / Model	Provider	Primary Area of Use	Primary Users
GitHub Copilot	GitHub / OpenAI	Code autocomplete, Express.js route scaffolding, React UI components, Prisma queries	Hoàng, Tấn, Chiến
ChatGPT (GPT-4o)	OpenAI	C4 diagram syntax drafting (Mermaid), Use-Case specification refinement, adversarial test scenario design, regex generation	Huy, Nghĩa, Chiến, Văn
Claude 3.5 Sonnet	Anthropic	Complex backend architectural reviews, Socket.io event boundary analysis, defect root-cause analysis	Chiến, Nghĩa
Spec Kit	Course Framework / LLM	Specification-driven development: automated drafting of specifications, user stories, task breakdowns, and contract markdown files	Whole Team

AI Tool / Model	Provider	Primary Area of Use	Primary Users
Google Gemini API (gemini-flash-latest)	Google Cloud	Runtime Application Feature: Chat red-flag safety analysis, ice-breaking mini-game question generation, and profile compatibility scoring	Embedded in LoveYou backend

3. Comprehensive AI Usage Log (PA1 – PA5)

Performed by: Nghĩa, Chiến, Văn | Reviewed by: Hoàng, Tấn | Edited by: Nghĩa

The table below catalogs major AI-assisted activities performed across all phases of the project:

Date	Phase	Member	Tool	Purpose / Prompt Objective	Generated Output & Human Verification
15/05/2026	PA1	Huy, Chiến	ChatGPT	Brainstorm core dating app features & target user personas in Vietnam	Outlined initial feature matrix and user personas; refined manually to fit Vietnamese context.
20/05/2026	PA1	Nghĩa	ChatGPT	Format PA1 Proposal Markdown and check grammatical tone	Formatted Markdown tables and headers in PA1-Group01.md.
05/06/2026	PA2	Huy	ChatGPT	Draft Use Case model and initial use case specifications	Generated initial behavioral steps for UC1–UC6; team manually added non-functional requirements.
10/06/2026	PA2	Chiến	ChatGPT	Formulate initial database schema structure in Prisma syntax	Produced first draft of schema.prisma; team added geospatial coordinates and indexes.
25/06/2026	PA3	Hoàng, Tấn	Spec Kit	Scaffold Module 001 (Auth) and 002 (Password Reset)	Generated spec.md, tasks.md, and contracts; adjusted token lifetimes and error codes.
02/07/2026	PA3	Chiến	GitHub Copilot	Scaffold Express auth middleware and bcrypt password hashing	Generated standard JWT validation boilerplate; manually added token revocation checks.
15/07/2026	PA3	Văn	Spec Kit	Generate initial test cases for authentication	Produced happy-path unit tests; team manually added edge cases for duplicate emails and malformed tokens.

Date	Phase	Member	Tool	Purpose / Prompt Objective	Generated Output & Human Verification
29/07/2026	PA4	Huy	ChatGPT	Revise Use-Case specifications and Actor hierarchy	Refined concise behavioral flows based on instructor feedback.
01/08/2026	PA4	Huy, Chiến	ChatGPT	Draft C4 Level 1 (Context), Level 2 (Container), and Level 3 (Components)	Generated Mermaid syntax; Chiến manually corrected container database connections and socket listeners.
02/08/2026	PA4	Nghĩa	ChatGPT	Draft C4 Deployment Diagram in Mermaid	Produced cloud deployment structure (Vercel, Render/Local, Neon PostgreSQL).
03/08/2026	PA4	Hoàng, Tấn	Spec Kit	Generate artifacts for Mini-Games (008) and Admin (009)	Generated specifications and task breakdowns; added schema for game session states.
04/08/2026	PA4	Hoàng, Tấn	GitHub Copilot	Frontend React components for Admin user management table	Generated Paginated user table and action buttons; integrated manually with backend APIs.
11/08/2026	PA5	Chiến	ChatGPT	Author adversarial test scenarios for Gemini red-flag analysis	Formulated synthetic phishing scams, financial extortion, and coercive control dialogues used in UC3 test cases.
12/08/2026	PA5	Chiến	Claude	Design property-based assertion strategy for non-deterministic AI outputs	Developed 4 assertion classes (Schema, Directional, Deterministic, Degradation) in TestPlan.md .
14/08/2026	PA5	Văn	ChatGPT	Brainstorm boundary test data for Vietnamese Citizen ID OCR and age filters	Generated edge-case test values (boundary ages 17/18, corrupted image types, oversized files) in UC6.
18/08/2026	PA5	Chiến	GitHub Copilot	Scaffold Supertest automated probe scripts for PA5 test execution	Generated Supertest request chains; team verified database assertions against Neon cloud.

Date	Phase	Member	Tool	Purpose / Prompt Objective	Generated Output & Human Verification
20/08/2026	PA5	Hoàng	ChatGPT	Analyze Vite production chunk size warning (BUG-17)	Suggested code-splitting and dynamic <code>React.lazy()</code> import strategies for post-release optimization.
22/08/2026	PA5	Nghĩa	ChatGPT	Review and refine grammar for <code>Reflection.md</code> and SDLC recommendations	Polished phrasing and ensured all 5 course reflection criteria were thoroughly articulated.

4. AI-Assisted Activities by Development Phase

Performed by: Chiến, Hoàng, Văn | Reviewed by: Nghĩa | Edited by: Chiến

4.1 Requirements & System Analysis (PA1 – PA2)

- **Feature Brainstorming:** Used LLMs to benchmark standard online dating capabilities (e.g., Tinder, Bumble) against localized Vietnamese dating dynamics, resulting in features such as Citizen ID verification and AI compatibility scoring.
- **Specification Structuring:** Drafted use-case steps, preconditions, and postconditions with AI prompt assistance, followed by manual adjustment to eliminate ambiguities.

4.2 Architectural Design & Diagrams (PA3 – PA4)

- **Mermaid Syntax Generation:** Generated syntax for C4 System Context, Container, Component, and Deployment diagrams. AI proved particularly proficient at generating valid Mermaid code from structured architectural bullet points.
- **Data Model Normalization:** Used AI to review entity relationships in `schema.prisma` (e.g., self-referencing `Match` models and polymorphic `UserBlock` associations).

4.3 Specification-Driven Development (Spec Kit)

- **Module Deconstruction:** Spec Kit automated the initial decomposition of feature requirements into markdown contracts (`contracts/*.md`) and actionable task lists (`tasks.md`) across 12 modules.
- **Contract Standardization:** Enforced consistent RESTful endpoint definitions, HTTP status conventions, and request/response payloads before backend controllers were coded.

4.4 Code Implementation (Frontend & Backend)

- **Boilerplate & Controller Scaffolding:** GitHub Copilot accelerated the authoring of standard Express route handlers, input validation middlewares, and React hook setups by roughly 40%.
- **Validation Regular Expressions:** Prompted AI to construct regex patterns for strict email validation, phone formats, and Citizen Identity Card (CCCD) numbers.

4.5 Quality Assurance & Test Case Design (PA5)

- **Adversarial Test Data Synthesis:** Created realistic ground-truth conversational dialogues (financial scam pitches, urgent OTP demands, manipulative gaslighting) to validate the functional accuracy of Gemini's `detectRedFlags()` feature.
- **Metamorphic AI Testing Logic:** Formulated metamorphic test cases where opposing user answer sets were asserted to score strictly lower than identical answer sets (`TC-GAME-09` vs. `TC-GAME-10`).
- **Defect Classification:** Assisted in structuring the 25 logged bugs in `BugReport.md`, ensuring all required attributes (ID, steps, expected vs. actual, severity, status) were complete.

5. Human Review, Verification & Quality Control

Performed by: Whole Team | **Reviewed by:** Nghĩa | **Edited by:** Nghĩa

The team maintained a strict **"Zero Blind Acceptance"** policy for all AI-assisted outputs. Every code snippet, diagram, or test case underwent rigorous human review through the following controls:

5.1 Verification Checklist for AI-Generated Outputs

1. **Compilation & Syntax Execution:** Code generated or suggested by AI was never committed without immediate local execution (`npm run dev`, `npx jest`).
2. **Contract Conformance:** API implementations were verified against the written markdown contracts in `src/specs/` to ensure request/response parity.
3. **Defensive Edge Case Check:** AI suggestions were systematically checked for missing boundary guards (e.g., checking if an unmatched occurred before socket emission).
4. **Security & Secrets Scrubbing:** Ensured AI-generated test fixtures and documentation never contained hardcoded secrets, database credentials, or live Gemini API keys.

5.2 Concrete Examples of AI Hallucinations & Human Corrections

Phase / File	AI Proposal / Generated Content	Issue / Hallucination Identified	Human Correction Applied
PA3 / Spec Kit (<code>002-password-reset</code>)	Generated test asserted HTTP 404 for unknown email in forgot-password flow.	Violates OWASP security principles by enabling account enumeration.	Corrected the contract to expect generic HTTP 200; logged code mismatch as <code>BUG-01</code> .
PA4 / Backend (<code>authMiddleware.js</code>)	Copilot suggested using deprecated Express 4 error-handling signature in Express 5.	Caused unhandled promise rejections during token verification failures.	Manually refactored to Express 5 async error handling standards.
PA4 / Architecture (<code>Architecture.md</code>)	ChatGPT generated a C4 Container diagram showing React connecting directly to PostgreSQL.	Architectural anti-pattern; violates 3-tier web architecture.	Corrected the Mermaid graph to route all database interactions exclusively through Express.js.

Phase / File	AI Proposal / Generated Content	Issue / Hallucination Identified	Human Correction Applied
PA5 / Testing (TestCases.md)	Spec Kit generated tests asserting exact response strings from Gemini AI.	Flaky tests; LLMs are non-deterministic and never return identical strings.	Re-authored into property-based assertions checking JSON schema, score bounds ($0 \leq s \leq 100$), and directional danger levels.
PA5 / Build (package.json)	AI scaffolding declared @payos/node in dependencies but failed to install it.	Caused Cannot find module runtime crash upon fresh clone (BUG-13).	Resolved via npm install and added dependency check step to verification workflow.

6. Impact Analysis & Academic Integrity Statement

Performed by: Nghĩa, Chiến | Reviewed by: Whole Team | Edited by: Nghĩa

6.1 Quantitative & Qualitative Impact

- Development Velocity:** AI assistance accelerated repetitive boilerplate generation and diagram drafting by an estimated 35%–40%, allowing the team to invest more time in test planning, defect investigation, and property testing for generative AI.
- Engineering Discipline:** Auditing AI outputs forced the team to adopt a more critical engineering mindset. Learning that auto-generated test cases often contain false assumptions trained team members to inspect specifications and code boundaries with heightened rigor.

6.2 Academic Integrity Affirmation

Group 01 confirms that all work presented in this final submission package is the original creation and intellectual responsibility of the project members. AI tools were employed strictly within the bounds of university academic guidelines as assistive utilities. Every architectural decision, code commit, test execution, defect analysis, and reflection represents the genuine learning and effort of the team.